

DPDP COMPLIANCE AUDIT

saas-copilot

Digital Personal Data Protection Act 2023



65/100

Grade C — Needs Work

Scanned: 24 March 2026

Files indexed: 45

Languages: FastAPI, SQLAlchemy, Pydantic, passlib, python-jose, uvicorn

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

The application collects user name, email, and password for authentication. The biggest DPDP risk is the storage and potential exposure of user credentials (password hash) and PII (email, name) without explicit consent mechanisms visible in the code.

Validation Summary Rule findings: 11 confirmed, 3 false positives suppressed • AI gaps: 2 dual-validated • Score: 65 (rules) → 65 (integrated)

Total findings: 13
HIGH: 4
MEDIUM: 3
INFO: 3
PASS: 1

Tech Stack: FastAPI, SQLAlchemy, Pydantic, passlib, python-jose
Risk Profile: The application collects user name, email, and password for authentication. The biggest DPDP risk is the storage and potential exposure of user credentials (password hash) and PII (email, name) without explicit consent mechanisms visible in the code.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	11/22 (50%)	WARN
Section 6(6)	8	0/8 (0%)	FAIL
Section 8(1)	18	18/18 (100%)	PASS
Section 8	10	10/10 (100%)	PASS
Section 8(3)	8	8/8 (100%)	PASS
Section 8(4)	8	0/8 (0%)	FAIL
Section 8(6)	8	6.8/8 (85%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	2.1/6 (35%)	FAIL
Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal		0/4 (0%)	FAIL

Estimated Total Remediation Effort: 2.6–9.1 engineering days

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Application Logs

PII fields: email, password
Signup.jsx → api.js

Flow 2: Application Logs

PII fields: email, password
Login.jsx → api.js

Rule	Section	Severity	File
CONSENT_MISSING_REPO_LEVEL	Section 6 — Consent	HIGH	REPO-WIDE
CONSENT_WITHDRAWAL_MISSING	Section 6(6) — Consent Withdrawal	HIGH	N/A
AUDIT_TRAIL_MISSING	Section 8(4) — Audit Trail	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
DATA_ACCESS_MISSING	Section 11 — Right to Access	MEDIUM	N/A
NO_BREACH_ALERTING	Section 8(6) — Breach	LOW	N/A
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
CONSENT_PRESENT	Section 6 — Consent	PASS	backend/app/api/auth.py
	Section 5 — Purpose Limitation	MEDIUM	
	Section 5 — Purpose Limitation	MEDIUM	

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (10–36 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Missing Repo Level — REPO-WIDE (4–16 hrs)
- Consent Withdrawal Missing (1–4 hrs)
- Audit Trail Missing (4–12 hrs)
- Grievance Officer Missing (1–4 hrs)

Week 2 — High Priority (6–22 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Data Access Missing (2–6 hrs)
- (2–8 hrs)
- (2–8 hrs)

Week 4 — Advisory (4–14 hours)

Address before next compliance review.

- No Breach Alerting (2–6 hrs)
- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 20–72 hours (2.5–9.0 engineering days)

Changes Since 2026-03-24

▲ **Score: 64.0 → 65 (+1.0 points)**

✓ 1 finding(s) resolved

■ 3 new finding(s) introduced

— 11 finding(s) unchanged

Resolved since last scan:

✓

New findings this scan:

- Password Not Hashed [HIGH]
- No Deletion Mechanism [MEDIUM]
- Retention Missing [MEDIUM]

Details

Finding 1 of 13

[Section 6 — Consent]

1. [HIGH] CONSENT_MISSING_REPO_LEVEL — CODEBASE-WIDE:27

Evidence snippet: The signup endpoint processes user data (email, password, name) but lacks any explicit consent collection or user agreement step.

Estimated fix time: 4–16 hours

No consent mechanism detected anywhere in the repository.

Risk:

The absence of a consent mechanism means the application is not obtaining explicit permission from users before processing their personal data, violating DPDP Section 6. This can lead to non-compliance penalties and loss of user trust.

Fix:

1. Implement a consent checkbox on the signup form (frontend) that users must tick to agree to terms and privacy policy.
2. Modify the `signup` endpoint in `backend/app/api/auth.py` to accept and store user consent status.
3. Create a new database field in the `User` model to store the consent status and timestamp.
4. Add a middleware or decorator to `backend/app/main.py` to check for consent on relevant API endpoints that process personal data.

DPDP:

Section 6 — Consent requires explicit, informed, and freely given consent for the processing of personal data.

Example:

```
python # backend/app/schemas/user.py from pydantic import BaseModel class
UserCreate(BaseModel): name: str email: str password: str consent: bool = False # Add consent
field # backend/app/api/auth.py @router.post('/signup') def signup(payload: UserCreate, db:
Session = Depends(get_db)): # ... existing code ... if not payload.consent: raise
HTTPException(status_code=400, detail='Consent is required') # ... rest of the code ...
```

Finding 2 of 13

[Section 6(6) — Consent Withdrawal]

2. [HIGH] CONSENT_WITHDRAWAL_MISSING — N / A:27

Evidence snippet: The signup endpoint processes user data but lacks any consent collection, and consequently, no withdrawal mechanism is present.

Estimated fix time: 1–4 hours

Consent is collected but no withdrawal mechanism detected. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent. Users must be able to revoke consent at any time.

Risk:

The absence of a consent withdrawal mechanism means users cannot easily revoke their consent for data processing as required by DPDP Section 6(6). This hinders user control over their data and can lead to non-compliance.

Fix:

1. Create a new API endpoint (e.g., `/auth/withdraw-consent`) in `backend/app/api/auth.py`.
2. Implement logic in the new endpoint to update the user's consent status to false in the database.
3. Ensure this withdrawal endpoint is accessible to authenticated users and is as easy to use as giving consent.
4. Update the frontend to provide a clear option for users to withdraw their consent, linking to the new API endpoint.

DPDP:

Section 6(6) — Consent withdrawal must be as easy as giving consent, allowing users to revoke consent at any time.

Example:

```
python # backend/app/api/auth.py @router.post('/withdraw-consent') def
withdraw_consent(current_user: User = Depends(get_current_user), db: Session =
Depends(get_db)): # Assuming get_current_user is a dependency that returns the logged-in user
current_user.consent = False current_user.consent_timestamp = datetime.utcnow() # Optional:
track withdrawal time db.commit() return {"message": "Consent withdrawn successfully"}
```

Finding 3 of 13

[Section 8(4) — Audit Trail]

3. [HIGH] AUDIT_TRAIL_MISSING — N / A**Estimated fix time:** 4–12 hours

No audit trail infrastructure detected. DPDP Section 8(4) requires complete records of personal data processing activities. No access logging, audit model, or change tracking found in the codebase.

Risk:

The lack of a comprehensive audit trail means there are no complete records of personal data processing activities, violating DPDP Section 8(4). This makes it difficult to track data access, modifications, or deletions, increasing the risk of unauthorized actions and hindering accountability.

Fix:

1. Create a dedicated `AuditLog` model in `backend/app/models/audit.py` to store details of data processing events (user, action, timestamp, data affected).
2. Implement a logging mechanism (e.g., using a decorator or middleware) in `backend/app/main.py` to capture relevant events (e.g., user creation, login, data modification) and record them in the `AuditLog` model.
3. Ensure the `log\_requests` utility in `app.utils.logger` is enhanced to capture and log PII-related operations.
4. Add specific logging for sensitive operations within `backend/app/api/auth.py` and other relevant API endpoints.

DPDP:

Section 8(4) — Requires organizations to maintain complete and accurate records of all personal data processing activities.

Example:

```
python # backend/app/models/audit.py from sqlalchemy import Column, Integer, String, DateTime,
ForeignKey from app.db.base_class import Base from datetime import datetime class
AuditLog(Base): __tablename__ = "audit_log" id = Column(Integer, primary_key=True, index=True)
user_id = Column(Integer, ForeignKey("user.id")) action = Column(String, index=True) timestamp
= Column(DateTime, default=datetime.utcnow) details = Column(String, nullable=True) #
backend/app/utils/logger.py (example modification) # ... existing imports ... from
app.models.audit import AuditLog from app.db.session import SessionLocal def
log_audit_event(user_id: int, action: str, details: str = ""): db = SessionLocal() log_entry =
AuditLog(user_id=user_id, action=action, details=details) db.add(log_entry) db.commit()
db.close()
```

Finding 4 of 13

[Section 13 — Grievance Redressal]

4. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A**Estimated fix time:** 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route ****and**** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The absence of a designated grievance officer means there is no clear point of contact for individuals to raise concerns or complaints regarding their personal data. This violates DPDP Section 13, potentially leading to unresolved issues, user dissatisfaction, and regulatory scrutiny.

Fix:

1. Appoint a specific individual within the organization to act as the Grievance Officer.
2. Clearly document the Grievance Officer's contact information (e.g., email address, phone number).
3. Publish the Grievance Officer's contact details prominently on the company's privacy policy page and/or website footer.
4. Create a dedicated endpoint or form on the website for users to submit grievances, which should be routed to the Grievance Officer.

DPDP:

Section 13 — Requires organizations to establish a mechanism for addressing grievances related to personal data processing.

Example:

```
python # frontend/src/components/Footer.jsx (example) function Footer() { return ( <footer>
<p> For any grievances regarding your personal data, please contact our Grievance Officer at:
<a href="mailto:grievance.officer@example.com">grievance.officer@example.com</a> </p> { /*
Other footer content */ } </footer> ); } export default Footer;
```

Finding 5 of 13

[Section 11 — Right to Access]

5. [MEDIUM] DATA_ACCESS_MISSING — N / A:37**Evidence snippet:** Includes routers for 'auth' and 'rag', but no specific endpoint for data access is defined here.**Estimated fix time:** 2–6 hours

No endpoint detected that allows authenticated users to retrieve their own personal data. Under DPDP Section 11, Data Principals have the right to access information about their personal data being processed.

Risk:

The absence of a data access endpoint means users cannot exercise their right to access their personal data, potentially leading to non-compliance and user dissatisfaction.

Fix:

1. Create a new FastAPI router and endpoint (e.g., GET /users/me/data) that retrieves the authenticated user's data from the database.
2. Ensure the endpoint is protected and only accessible by the authenticated user.
3. Return the user's personal data in a structured format (e.g., JSON).
4. Document this new endpoint in the API documentation.

DPDP:

Section 11 — Right to Access

Example:

```
from fastapi import APIRouter, Depends from sqlalchemy.orm import Session from app.db.session import get_db from app.models.user import User from app.core.security import get_current_user\n# Assuming you have a function to get current user data_router = APIRouter(prefix="/users/me", tags=["User Data"]) @data_router.get("/data") def get_my_data(current_user: User = Depends(get_current_user), db: Session = Depends(get_db)): # Fetch user data, excluding sensitive fields like password_hash user_data = db.query(User).filter(User.id == current_user.id).first() if not user_data: return {"message": "User not found"} # Should not happen if get_current_user works # Return a subset of user data, e.g., name, email return {"name": user_data.name, "email": user_data.email}
```


Finding 6 of 13

[Section 8(6) — Breach]

6. [LOW] NO_BREACH_ALERTING — N / A:37**Evidence snippet:** Includes routers for 'auth' and 'rag', but no explicit breach alerting mechanism is configured.**Estimated fix time:** 2–6 hours

Structured logging detected but no breach notification mechanism found. DPDP Section 8(6) requires notifying affected individuals and the Authority without undue delay when a breach occurs. Logging alone is insufficient — implement automated alerting (Sentry, PagerDuty, or equivalent) with escalation to a compliance team.

Risk:

The lack of a breach notification mechanism means that in the event of a data breach, affected individuals and the Authority may not be promptly informed, violating DPDP Section 8(6).

Fix:

1. Integrate a third-party error tracking and alerting service (e.g., Sentry, Datadog, PagerDuty) into the application.
2. Configure the service to monitor for critical errors, security-related exceptions, and potential data exfiltration patterns.
3. Set up automated notifications to a designated compliance team or security contact upon detection of a potential breach.
4. Ensure the alerting system includes escalation policies and clear response procedures.

DPDP:

Section 8(6) — Breach notification

Example:

```
import sentry_sdk
sentry_sdk.init( dsn="YOUR_SENTRY_DSN", # Set traces_sample_rate to 1.0 to
capture 100% of transactions for performance monitoring. # We recommend adjusting this value
in production. traces_sample_rate=1.0, # If you wish to associate users to events, uncomment
the following line # and add a user identifier into the event's context. #
send_default_pii=True ) # In your FastAPI app setup (e.g., backend/app/main.py): #
app.add_middleware(SentryMiddleware) # Example of capturing an exception that might indicate a
breach: try: # ... sensitive operation ... except Exception as e:
sentry_sdk.capture_exception(e) raise HTTPException(status_code=500, detail="An error
occurred")
```

Finding 7 of 13

[Section 11 — Data Portability]

7. [LOW] DATA_PORTABILITY_MISSING — N / A:37**Evidence snippet:** Includes routers for 'auth' and 'rag', but no specific endpoint for data export or portability is defined.**Estimated fix time:** 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The absence of a data portability endpoint prevents users from obtaining their personal data in a machine-readable format, failing to meet the requirements of DPDP Section 11.

Fix:

1. Create a new FastAPI router and endpoint (e.g., GET /users/me/export) for data portability.
2. This endpoint should allow an authenticated user to request their data in a common machine-readable format (e.g., CSV, JSON).
3. Implement logic to fetch the user's data, format it appropriately, and return it as a downloadable file.
4. Ensure the endpoint is secured and accessible only by the authenticated user.

DPDP:

Section 11 — Data Portability

Example:

```
from fastapi import APIRouter, Depends, Response from sqlalchemy.orm import Session from app.db.session import get_db from app.models.user import User from app.core.security import get_current_user import csv import io portability_router = APIRouter(prefix="/users/me", tags=["User Data Portability"]) @portability_router.get("/export") def export_my_data(current_user: User = Depends(get_current_user), db: Session = Depends(get_db)): user_data = db.query(User).filter(User.id == current_user.id).first() if not user_data: return {"message": "User not found"} # Prepare data for CSV export output = io.StringIO() writer = csv.writer(output) writer.writerow(["id", "name", "email"]) writer.writerow([user_data.id, user_data.name, user_data.email]) return Response(content=output.getvalue(), media_type="text/csv", headers={"Content-Disposition": "attachment; filename=user_data.csv"})
```

Finding 8 of 13

[Section 5 — Notice]

8. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 9 of 13

[Section 5 — Purpose Limitation]

9. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 10 of 13

[Section 16 — Cross-Border Transfer]

10. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 11 of 13

[Section 6 — Consent]

11. [PASS] CONSENT_PRESENT — backend / app / api / auth.py

Auth enforcement pattern detected inline — consent likely handled at registration.

Finding 12 of 13

[Section 5 — Purpose Limitation]

12. [MEDIUM] — CODEBASE-WIDE

Excessive data collection for user name

Finding 13 of 13

[Section 5 — Purpose Limitation]

13. [MEDIUM] — CODEBASE-WIDE

PII redaction missing in request logs

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] Excessive data collection for user name

Confidence: 67%

Section 5 — Purpose Limitation

The ``backend/app/schemas/user.py`` defines ``name`` as a required field for user creation, and ``backend/app/api/auth.py`` handles its collection. However, the core RAG functionality described in the data flows does not explicitly require a user's full name.

Recommendation: Review ``backend/app/schemas/user.py`` and ``backend/app/api/auth.py`` to determine if the ``name`` field is strictly necessary for the application's core purpose. If not, make it optional or remove it to adhere to data minimization principles.

[AI-INFERRED] PII redaction missing in request logs

Confidence: 53%

Section 5 — Purpose Limitation

The ``log_requests`` middleware in ``utils/logger.py`` logs request details. The repository context indicates it 'does not appear to redact PII if it were present in URLs or request bodies,' posing a risk of accidental PII logging.

Recommendation: Implement PII redaction logic within the ``log_requests`` middleware in ``utils/logger.py``. This should parse request URLs, headers, and potentially bodies to identify and mask or remove sensitive personal data before logging.